ฐานข้อมูลขั้นสูง · GRAPH DATABASE

Dog Recommendation Graph ด้วย Neo4j

ระบบแนะนำสายพันธุ์สุนัขให้ผู้เลี้ยง จากสุนัขที่เพื่อนเลี้ยงและลักษณะที่ตรงกับความต้องการ โดยใช้ Cypher บนฐานข้อมูลกราฟ

ชื่อ-สกุล .....

รหัสนักศึกษา .....

เชค .....

วันที่ส่ง .....

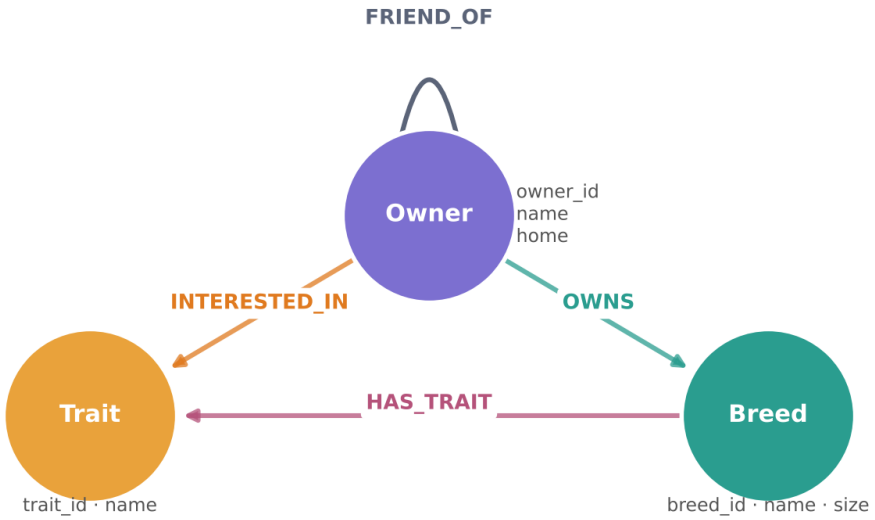
1 ออกแบบโมเดลกราฟ

โจทย์คือแนะนำสายพันธุ์สุนัขให้ผู้เลี้ยงคนหนึ่ง โดยพิจารณา 2 อย่าง คือ
เพื่อนของเขาเลี้ยงสุนัขพันธุ์อะไร และ **สายพันธุ์นั้นมีลักษณะตรงกับที่เขาต้องการหรือไม่**
จึงออกแบบข้อมูลเป็น 3 Node และ 4 Relationship

ตารางที่ 1 องค์ประกอบของกราฟ

องค์ประกอบ	Property / ทิศทาง	เหตุผลที่ออกแบบแบบนี้
:Owner	owner_id, name, home	ผู้เลี้ยง เป็นทั้งคนที่ได้รับคำแนะนำและเป็นเพื่อนของคนอื่น
:Breed	breed_id, name, size	สายพันธุ์สุนัข คือสิ่งที่ต้องการแนะนำ
:Trait	trait_id, name	ลักษณะของสุนัข ใช้ร่วมกันได้หลายคนและหลายพันธุ์ จึงแยกเป็น Node แทนการเก็บเป็นข้อความ
FRIEND_OF	Owner → Owner	เส้นทางหลักของคำแนะนำ (เก็บทิศเดียว แต่ query แบบไม่สนทิศ)
OWNS	Owner → Breed	ผู้เลี้ยงแต่ละคนเลี้ยงสุนัขพันธุ์อะไร
INTERESTED_IN	Owner → Trait	ลักษณะที่ผู้เลี้ยงต้องการ
HAS_TRAIT	Breed → Trait	ลักษณะเด่นของสายพันธุ์ ใช้จับคู่กับความต้องการ

ลักษณะ (Trait) ที่ใช้มี 4 แบบ ได้แก่ **Active** ขี้เล่นและชอบออกกำลังกาย, **Apartment** เลี้ยงในคอนโดได้, **Family** เข้ากับเด็กและครอบครัว และ **Low-shedding** ขนร่วงน้อย



รูปที่ 1 โครงสร้าง (schema) ของกราฟ

2 สร้าง Constraint และข้อมูลตัวอย่าง

ก่อนใส่ข้อมูล สร้าง Constraint ให้รหัสของแต่ละ Node ห้ามซ้ำ ป้องกันการสร้าง Node ซ้ำโดยไม่ตั้งใจ

```
CREATE CONSTRAINT owner_id_unique IF NOT EXISTS
FOR (o:Owner) REQUIRE o.owner_id IS UNIQUE;

CREATE CONSTRAINT breed_id_unique IF NOT EXISTS
FOR (b:Breed) REQUIRE b.breed_id IS UNIQUE;
```

```
CREATE CONSTRAINT trait_id_unique IF NOT EXISTS
FOR (t:Trait) REQUIRE t.trait_id IS UNIQUE;
```

จากนั้นสร้างข้อมูลตัวอย่าง ได้แก่ ผู้เลี้ยง 7 คน สายพันธุ์ 6 แบบ และลักษณะ 4 แบบ คนที่ใช้ทดสอบคือ Ploy (O001) ซึ่งอยู่คอนโดและเลี้ยง Pomeranian อยู่แล้ว ต้องการสุนัขที่เลี้ยงในคอนโดได้ (Apartment) และขนร่วงน้อย (Low-shedding) โดยมีเพื่อน 4 คน

```
CREATE
  // ผู้เลี้ยง 7 คน
  (o1:Owner {owner_id:'O001', name:'Ploy', home:'Condo'}),
  (o2:Owner {owner_id:'O002', name:'Bank', home:'House'}),
  (o3:Owner {owner_id:'O003', name:'Ice',  home:'House'}),
  (o4:Owner {owner_id:'O004', name:'Nan',  home:'Condo'}),
  (o5:Owner {owner_id:'O005', name:'Pete', home:'House'}),
  (o6:Owner {owner_id:'O006', name:'Jane', home:'Townhouse'}),
  (o7:Owner {owner_id:'O007', name:'Tong', home:'Condo'}),
  // สายพันธุ์ 6 แบบ
  (b1:Breed {breed_id:'B01', name:'Golden Retriever', size:'Large'}),
  (b2:Breed {breed_id:'B02', name:'Shiba Inu',        size:'Medium'}),
  (b3:Breed {breed_id:'B03', name:'Poodle',           size:'Small'}),
  (b4:Breed {breed_id:'B04', name:'Corgi',            size:'Medium'}),
  (b5:Breed {breed_id:'B05', name:'Pomeranian',       size:'Small'}),
  (b6:Breed {breed_id:'B06', name:'Labrador',         size:'Large'}),
  // ลักษณะ 4 แบบ
  (t1:Trait {trait_id:'T01', name:'Active'}),
  (t2:Trait {trait_id:'T02', name:'Apartment'}),
  (t3:Trait {trait_id:'T03', name:'Family'}),
  (t4:Trait {trait_id:'T04', name:'Low-shedding'}),
  // เพื่อน (เก็บทิศเดียว ตอน query มองแบบไม่สนทิศ)
  (o1)-[:FRIEND_OF]->(o2), (o1)-[:FRIEND_OF]->(o3),
  (o1)-[:FRIEND_OF]->(o4), (o1)-[:FRIEND_OF]->(o5),
  (o2)-[:FRIEND_OF]->(o3), (o3)-[:FRIEND_OF]->(o5),
  (o4)-[:FRIEND_OF]->(o6), (o6)-[:FRIEND_OF]->(o7),
  // ใครเลี้ยงพันธุ์อะไร
  (o1)-[:OWNS]->(b5),
  (o2)-[:OWNS]->(b1), (o2)-[:OWNS]->(b6),
  (o3)-[:OWNS]->(b1), (o3)-[:OWNS]->(b5),
  (o4)-[:OWNS]->(b3),
  (o5)-[:OWNS]->(b6), (o5)-[:OWNS]->(b2),
  (o6)-[:OWNS]->(b3), (o6)-[:OWNS]->(b4),
  (o7)-[:OWNS]->(b4),
  // ความต้องการของผู้เลี้ยง
  (o1)-[:INTERESTED_IN]->(t2), (o1)-[:INTERESTED_IN]->(t4),
  (o2)-[:INTERESTED_IN]->(t1),
  (o3)-[:INTERESTED_IN]->(t3),
  (o4)-[:INTERESTED_IN]->(t4),
  (o5)-[:INTERESTED_IN]->(t1),
  (o6)-[:INTERESTED_IN]->(t3), (o6)-[:INTERESTED_IN]->(t2),
  (o7)-[:INTERESTED_IN]->(t2),
  // ลักษณะเด่นของแต่ละสายพันธุ์
  (b1)-[:HAS_TRAIT]->(t1), (b1)-[:HAS_TRAIT]->(t3),
  (b2)-[:HAS_TRAIT]->(t1),
  (b3)-[:HAS_TRAIT]->(t2), (b3)-[:HAS_TRAIT]->(t4),
  (b4)-[:HAS_TRAIT]->(t3), (b4)-[:HAS_TRAIT]->(t2),
  (b5)-[:HAS_TRAIT]->(t2),
  (b6)-[:HAS_TRAIT]->(t1), (b6)-[:HAS_TRAIT]->(t3);
```

3 ตรวจสอบจำนวนข้อมูล

นับ Node และ Relationship ที่สร้างแล้วเทียบกับที่ออกแบบไว้ ได้ Node รวม 17 และ Relationship รวม 38 ตรงตามที่ออกแบบ

```
MATCH (n)
RETURN labels(n)[0] AS label, count(*) AS total
ORDER BY label;
```

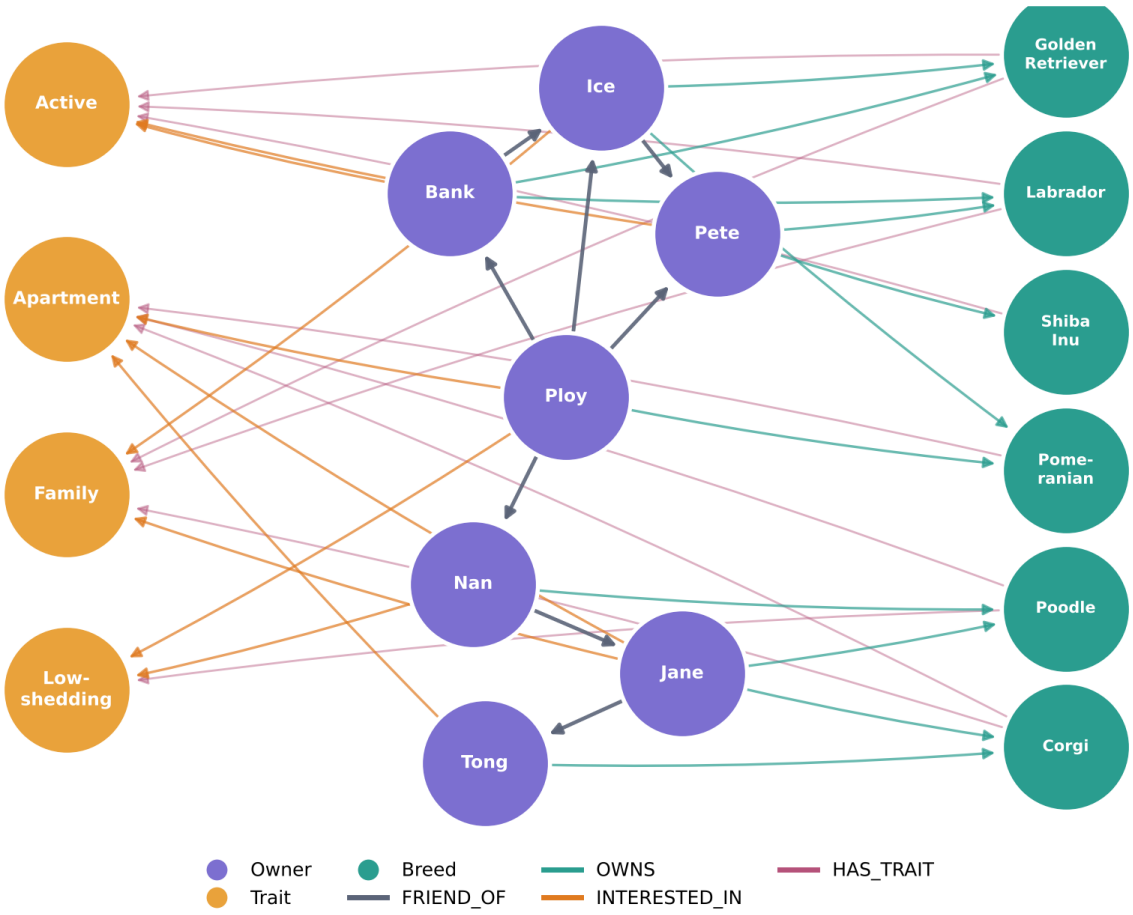
ผลลัพธ์ (3 แถว)

label	total
Breed	6
Owner	7
Trait	4

```
MATCH ()-[r]->()
RETURN type(r) AS relationship, count(*) AS total
ORDER BY relationship;
```

ผลลัพธ์ (4 แถว)

relationship	total
FRIEND_OF	8
HAS_TRAIT	10
INTERESTED_IN	9
OWNS	11



รูปที่ 2 กราฟทั้งหมดหลังสร้างข้อมูล (Trait ซ้าย, Owner กลาง, Breed ขวา)

4 ดูว่าเพื่อนของ Ploy เลี้ยงสุนัขพันธุ์อะไรบ้าง

เดิน 2 ทอดจาก Ploy ไปหาเพื่อน แล้วต่อไปยังสายพันธุ์ที่เพื่อนเลี้ยง ที่ FRIEND_OF ใช้เครื่องหมาย - โดยไม่ใส่ลูกศร เพราะเก็บความเป็นเพื่อนไว้ทิศเดียว

```
MATCH (me:Owner {owner_id:'O001'})-[:FRIEND_OF]-(friend:Owner)
      -[:OWNS]->(breed:Breed)
RETURN friend.name AS friend_name,
       breed.name  AS breed_name
ORDER BY friend_name, breed_name;
```

ผลลัพธ์ (7 แถว)

friend_name	breed_name
Bank	Golden Retriever
Bank	Labrador
Ice	Golden Retriever
Ice	Pomeranian
Nan	Poodle
Pete	Labrador
Pete	Shiba Inu

5 นับจำนวนเพื่อนที่เลี้ยงแต่ละสายพันธุ์

เปลี่ยนมุมมองจากรายคนเป็นรายสายพันธุ์ ใช้ `count(DISTINCT friend)` เป็นคะแนนเพื่อน และ `collect` เพื่อดูว่าเป็นเพื่อนคนไหนบ้าง

```
MATCH (me:Owner {owner_id:'O001'})-[:FRIEND_OF]-(friend:Owner)
      -[:OWNS]->(breed:Breed)
RETURN breed.breed_id AS breed_id,
       breed.name     AS breed_name,
       count(DISTINCT friend)        AS friend_score,
       collect(DISTINCT friend.name) AS friends
ORDER BY friend_score DESC, breed_name;
```

ผลลัพธ์ (5 แถว)

breed_id	breed_name	friend_score	friends
B01	Golden Retriever	2	Bank, Ice
B06	Labrador	2	Bank, Pete
B05	Pomeranian	1	Ice
B03	Poodle	1	Nan
B02	Shiba Inu	1	Pete

> สังเกตว่า Golden Retriever กับ Labrador ได้ 2 คะแนนเท่ากัน และ **Pomeranian ติดมาด้วยทั้งที่ Ploy เลี้ยงอยู่แล้ว** จึงต้องกรองออกในขั้นถัดไป

6 ตัดสายพันธุ์ที่ Ploy เลี้ยงอยู่แล้ว

ใช้ `WHERE NOT EXISTS` ตรวจว่าระหว่าง Ploy กับสายพันธุ์นั้นยังไม่มีเส้น OWNS ถ้ามีแล้วก็ไม่ต้องแนะนำซ้ำ

```
MATCH (me:Owner {owner_id:'O001'})-[:FRIEND_OF]-(friend:Owner)
      -[:OWNS]->(breed:Breed)
WHERE NOT EXISTS { (me)-[:OWNS]->(breed) }
RETURN breed.name AS breed_name,
       count(DISTINCT friend) AS friend_score
ORDER BY friend_score DESC, breed_name;
```

ผลลัพธ์ (4 แถว)

breed_name	friend_score
Golden Retriever	2
Labrador	2
Poodle	1
Shiba Inu	1

7 เพิ่มคะแนนจากลักษณะที่ตรงกับความต้องการ

ถ้าสายพันธุ์มีลักษณะที่ Ploy ต้องการ ให้บวกเพิ่มลักษณะละ 1 คะแนน โดยใช้ `WITH` รวมคะแนนเพื่อนเป็นรายสายพันธุ์ก่อน แล้วค่อย `OPTIONAL MATCH` หาลักษณะที่ตรงกัน ถ้าไม่รวมก่อน

แถวจะคูณกันทำให้นับผิด ส่วน OPTIONAL ทำให้สายพันธุ์ที่ไม่มีลักษณะตรงกันยังอยู่ในผลลัพธ์ (โบนัส = 0)

```
MATCH (me:Owner {owner_id:'O001'})-[:FRIEND_OF]-(friend:Owner)
      -[:OWNS]->(breed:Breed)
WHERE NOT EXISTS { (me)-[:OWNS]->(breed) }
WITH me, breed, count(DISTINCT friend) AS friend_score   // รวมเป็นรายสายพันธุ์ก่อน

OPTIONAL MATCH (me)-[:INTERESTED_IN]->(t:Trait)<-[:HAS_TRAIT]-(breed)
WITH breed, friend_score, count(t) AS trait_bonus        // ลักษณะที่ตรง 1 อย่าง = +1

RETURN breed.name AS breed_name,
       friend_score,
       trait_bonus,
       friend_score + trait_bonus AS total_score
ORDER BY total_score DESC, breed_name;
```

ผลลัพธ์ (4 แถว)

breed_name	friend_score	trait_bonus	total_score
Poodle	1	2	3
Golden Retriever	2	0	2
Labrador	2	0	2
Shiba Inu	1	0	1

> **ผลที่ได้:** ถ้าดูแค่เพื่อน Golden Retriever ได้อันดับ 1 แต่เมื่อรวมลักษณะที่ต้องการ Poodle ได้โบนัส 2 คะแนน (Apartment และ Low-shedding) คะแนนรวมเป็น 3 จึงขึ้นอันดับ 1 ซึ่งเหมาะกับ Ploy ที่อยู่คอนโดมากกว่าสุนัขตัวใหญ่

8 แสดงเหตุผลของคำแนะนำ

คะแนนอย่างเดียวไม่บอกที่มา จึงเก็บชื่อเพื่อนและลักษณะที่ตรงกันไว้เป็นลิสต์ ผู้ใช้จะเห็นว่าทำไมระบบจึงแนะนำสายพันธุ์นี้

```
MATCH (me:Owner {owner_id:'O001'})-[:FRIEND_OF]-(friend:Owner)
      -[:OWNS]->(breed:Breed)
WHERE NOT EXISTS { (me)-[:OWNS]->(breed) }
WITH me, breed, collect(DISTINCT friend.name) AS friends

OPTIONAL MATCH (me)-[:INTERESTED_IN]->(t:Trait)<-[:HAS_TRAIT]-(breed)
WITH breed, friends, collect(t.name) AS matched_traits

RETURN breed.name AS breed_name,
       friends,
       matched_traits,
       size(friends) + size(matched_traits) AS total_score
ORDER BY total_score DESC, breed_name;
```

ผลลัพธ์ (4 แถว)

breed_name	friends	matched_traits	total_score
Poodle	[Nan]	[Apartment, Low-shedding]	3

breed_name	friends	matched_traits	total_score
Golden Retriever	[Bank, Ice]	[]	2
Labrador	[Bank, Pete]	[]	2
Shiba Inu	[Pete]	[]	1

9 เขียนผลลัพธ์กลับเข้ากราฟ (RECOMMENDED)

เก็บคำแนะนำเป็น Relationship ใหม่ชื่อ RECOMMENDED พร้อมคะแนน
เพื่อเรียกใช้ซ้ำได้โดยไม่ต้องคำนวณใหม่ ใช้ `MERGE` เพื่อให้รันซ้ำแล้วไม่เกิดเส้นซ้ำ

```
MATCH (me:Owner {owner_id:'O001'})-[:FRIEND_OF]-(friend:Owner)
      -[:OWNS]->(breed:Breed)
WHERE NOT EXISTS { (me)-[:OWNS]->(breed) }
WITH me, breed, count(DISTINCT friend) AS friend_score

OPTIONAL MATCH (me)-[:INTERESTED_IN]->(t:Trait)<-[:HAS_TRAIT]-(breed)
WITH me, breed, friend_score, count(t) AS trait_bonus

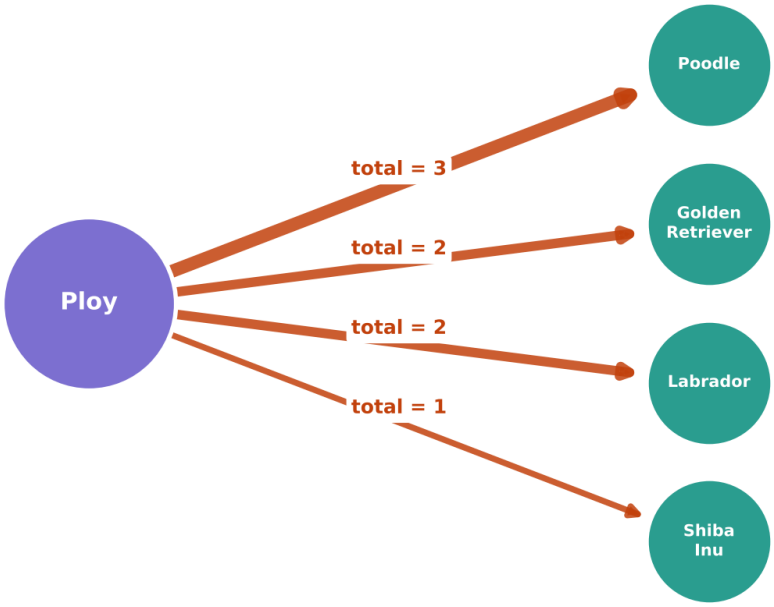
MERGE (me)-[r:RECOMMENDED]->(breed)
SET r.friend_score = friend_score,
    r.trait_bonus  = trait_bonus,
    r.total        = friend_score + trait_bonus
RETURN me.name AS owner, breed.name AS breed, r.total AS total
ORDER BY total DESC, breed;
```

ผลลัพธ์ (4 แถว)

owner	breed	total
Ploy	Poodle	3
Ploy	Golden Retriever	2
Ploy	Labrador	2
Ploy	Shiba Inu	1

ดูผลเป็นกราฟด้วยคำสั่งนี้

```
MATCH p = (:Owner {owner_id:'O001'})-[:RECOMMENDED]->(:Breed)
RETURN p;
```



รูปที่ 3 เส้น RECOMMENDED จาก Ploy ไปยังสายพันธุ์ที่แนะนำ (เส้นหนา = คะแนนสูง)

10 ทำเป็นฟังก์ชัน Python เพื่อใช้กับผู้เลี้ยงคนอื่น

นำ query ข้างต้นมาห่อเป็นฟังก์ชันใน Google Colab รับ `owner_id` แล้วคืนผลเป็นตาราง pandas

```
!pip -q install neo4j pandas
from neo4j import GraphDatabase
import pandas as pd

URI      = "neo4j+s://xxxxxxxx.databases.neo4j.io"
PASSWORD = "********"
DATABASE = "neo4j"
driver = GraphDatabase.driver(URI, auth=("neo4j", PASSWORD))

def recommend_breeds(owner_id):
    query = """
    MATCH (me:Owner {owner_id:$oid})-[:FRIEND_OF]-(friend:Owner)
          -[:OWNS]->(breed:Breed)
    WHERE NOT EXISTS { (me)-[:OWNS]->(breed) }
    WITH me, breed, count(DISTINCT friend) AS friend_score

    OPTIONAL MATCH (me)-[:INTERESTED_IN]->(t:Trait)<-[:HAS_TRAIT]-(breed)
    WITH breed, friend_score, count(t) AS trait_bonus

    RETURN breed.name AS breed,
           friend_score,
           trait_bonus,
           friend_score + trait_bonus AS total_score
    ORDER BY total_score DESC, breed
    """
    result = driver.execute_query(query, oid=owner_id, database_=DATABASE)
    return pd.DataFrame([r.data() for r in result.records])

recommend_breeds("O003")
recommend_breeds("O007")
```

recommend_breeds("O003") · Ice ผลลัพธ์ (2 แถว)

breed	friend_score	trait_bonus	total_score
Labrador	2	1	3
Shiba Inu	1	0	1

recommend_breeds("O007") · Tong ผลลัพธ์ (1 แถว)

breed	friend_score	trait_bonus	total_score
Poodle	1	1	2

Ice มีเพื่อนคือ Bank และ Pete ซึ่งเลี้ยง Labrador ทั้งคู่ และ Labrador มีลักษณะ Family ตรงกับที่ Ice ต้องการ คะแนนจึงเป็น 3 ส่วน Golden Retriever และ Pomeranian ถูกตัดออกเพราะ Ice เลี้ยงอยู่แล้ว สำหรับ Tong มีเพื่อนเพียงคนเดียวคือ Jane จึงได้คำแนะนำเฉพาะ Poodle (Corgi ถูกตัดเพราะ Tong เลี้ยงอยู่แล้ว) และ Poodle เลี้ยงในคอนโดได้ตรงกับที่ Tong ต้องการ คะแนนรวมจึงเป็น 2

11 สรุป

ระบบคิดคะแนนจาก 2 เส้นทาง คือ **เพื่อน → สายพันธุ์** (Owner–FRIEND_OF–Owner–OWNS–Breed) และ **ความต้องการ → สายพันธุ์** (Owner–INTERESTED_IN–Trait–HAS_TRAIT–Breed) เมื่อรวมกันแล้วได้คำแนะนำที่เหมาะกับแต่ละคนมากกว่าการนับเพื่อนอย่างเดียว ในกรณีของ Ploy ถ้านับเฉพาะเพื่อน Golden Retriever จะได้อันดับ 1 แต่เมื่อรวมลักษณะที่ต้องการ Poodle ได้โบนัสจาก Apartment และ Low-shedding จึงขึ้นเป็นอันดับ 1 ซึ่งเหมาะกับ Ploy ที่อยู่คอนโดมากกว่า

ข้อดีของ Graph Database คือเขียน query ตามเส้นทางความสัมพันธ์ได้โดยตรง ไม่ต้อง JOIN หลายตาราง และเพิ่มเงื่อนไขใหม่ได้ง่ายโดยไม่ต้องแก้โครงสร้างข้อมูล เช่น ถ้าไม่ต้องการแนะนำสุนัขตัวใหญ่ให้คนที่อยู่คอนโด ก็ใช้ Property `home` ของ Owner และ `size` ของ Breed เพิ่มเป็นเงื่อนไขใน query ได้ทันที